Xlib

Nathan Warner

Northern Illinois
University

Computer Science
Northern Illinois University
United States

Contents

1. Introduction

- **The X Window System (X11)**: The X Window System, often called X11, is the general windowing system design and protocol. It defines things like:
  - how graphical programs communicate with a display server,
  - how windows are created, moved, resized, and drawn into,
  - how keyboard and mouse input is delivered,
  - how clients and the display server exchange messages.

  There is a distinction between a specification/standard and a concrete program that implements it.
- **X.Org display server**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **X clients**: The X.Org display server, usually called Xorg, is an actual open-source program that implements the X11 server side of that system. It is the software process that runs on your machine and speaks the X11 protocol to graphical applications
- **Windowing system**: A windowing system is the overall graphical environment model that lets programs use windows, input devices, and the screen in a coordinated way.
- **Display server**: A display server is the actual program that sits between graphical applications and the hardware/display system.
  - **Windowing system:** X Window System / X11
  - **Display server:** X.Org Server

  A display server is responsible for managing access to the screen, keyboard, mouse, touchpad, and sometimes GPU/display hardware. Graphical applications usually do not draw directly to the physical screen. Instead, they communicate with the display server.

  A windowing system is broader. It is the architecture that defines how graphical applications, windows, input, drawing, and the display server/compositor interact.

  It includes concepts such as:
  - windows;
  - graphical clients;
  - input events;
  - drawing surfaces;
  - focus;
  - stacking order;
  - protocols for communication between apps and the graphical environment.
- **Intro**: The X Window System is a network-transparent window system that was designed at MIT. X display servers run on computers with either monochrome or color bitmap display hardware. The server distributes user input to and accepts output requests from various client programs located either on the same machine or elsewhere in the network. Xlib is a C subroutine library that application programs (clients) use to interface with the window system by means of a stream connection. Although a client usually runs on the same machine as the X server it is talking to, this need not be the case.

Note: Monochrome means **one color**. It describes any visual, design, or medium that is constructed using only a single base hue along with its various tints, tones, and shades. While black-and-white is the most common example, a monochrome palette can be built entirely around any color

- **Network-transparent**: In general, network-transparent means that a system lets you use some resource over a network as if it were local, while hiding most of the networking details from the user or program.
- **Color bitmap**: A color bitmap is an image represented as a rectangular grid of pixels, where each pixel stores color information. A color bitmap uses more bits per pixel so that each pixel can represent many possible colors. In a common 24-bit RGB color bitmap, each pixel stores three color components:

  $$(R, G, B).$$

  Each component uses 8 bits, so each ranges from 0 to 255.
- **Screens and Display**: The X Window System supports one or more screens containing overlapping windows or subwindows. A screen is a physical monitor and hardware that can be color, grayscale, or monochrome. There can be multiple screens for each display or workstation. A single X server can provide display services for any number of screens. A set of screens for a single user with one keyboard and one pointer (usually a mouse) is called a **display**.
- **Window hierarchy**: All the windows in an X server are arranged in strict hierarchies. At the top of each hierarchy is a root window, which covers each of the display screens. Each root window is partially or completely covered by child windows. All windows, except for root windows, have parents. There is usually at least one window for each application program. Child windows may in turn have their own children. In this way, an application program can create an arbitrarily deep tree on each screen. X provides graphics, text, and raster operations for windows.

  A child window can be larger than its parent. That is, part or all of the child window can extend beyond the boundaries of the parent, but all output to a window is clipped by its parent.

  That means X will only actually display the part of the child window that lies inside the parent window. Anything outside the parent's visible rectangle is not drawn on the screen.

  Children of a window have overlapping locations, one of the children is considered to be on top of or raised over the others, thus obscuring them. Output to areas covered by other windows is suppressed by the window system unless the window has backing store.

  Child windows obscure their parents, and graphic operations in the parent window usually are clipped by the children
- **Border**: A window has a border zero or more pixels in width, which can be any pattern (pixmap) or solid color you like. A window usually but not always has a background pattern, which will be repainted by the window system when uncovered.
- **Coordinates**: Each window and pixmap has its own coordinate system. The coordinate system has the X axis horizontal and the Y axis vertical with the origin [0, 0] at the upper-left corner. Coordinates are integral, in terms of pixels, and coincide with pixel centers. For a window, the origin is inside the border at the inside, upper-left corner.

- **Expose event**: X does not guarantee to preserve the contents of windows. When part or all of a window is hidden and then brought back onto the screen, its contents may be lost. The server then sends the client program an Expose event to notify it that part or all of the window needs to be repainted. Programs must be prepared to regenerate the contents of windows on demand.
- **Drawables**: X also provides off-screen storage of graphics objects, called pixmaps. Single plane (depth 1) pixmaps are sometimes referred to as bitmaps. Pixmaps can be used in most graphics functions interchangeably with windows and are used in various graphics operations to define patterns or tiles. Windows and pixmaps together are referred to as drawables.
- **Xlib function requests**: Most of the functions in Xlib just add requests to an output buffer. These requests later execute asynchronously on the X server. Functions that return values of information stored in the server do not return (that is, they block) until an explicit reply is received or an error occurs. You can provide an error handler, which will be called when the error is reported.
- **XSync**: If a client does not want a request to execute asynchronously, it can follow the request with a call to XSync, which blocks until all previously buffered asynchronous events have been sent and acted on. As an important side effect, the output buffer in Xlib is always flushed by a call to any function that returns a value from the server or waits for input.
- **Resource ID**: Many Xlib functions will return an integer resource ID, which allows you to refer to objects stored on the X server. These can be of type **Window, Font, Pixmap, Colormap, Cursor, and GContext**, as defined in the file **<X11/X.h>**. These resources are created by requests and are destroyed (or freed) by requests or when connections are closed. Most of these resources are potentially sharable between applications, and in fact, windows are manipulated explicitly by window manager programs. Fonts and cursors are shared automatically across multiple screens. Fonts are loaded and unloaded as needed and are shared by multiple clients. Fonts are often cached in the server. Xlib provides no support for sharing graphics contexts between applications
- **Client programs and events**: Client programs are informed of events. Events may either be side effects of a request (for example, restacking windows generates Expose ev ents) or completely asynchronous (for example, from the keyboard). A client program asks to be informed of events. Because other applications can send events to your application, programs must be prepared to handle (or ignore) events of all types.

  Input events (for example, a key pressed or the pointer moved) arrive asynchronously from the server and are queued until they are requested by an explicit call (for example, **XNextEvent** or **XWindowEvent**). In addition, some library functions (for example, **XRaiseWindow**) generate Expose and **ConfigureRequest** events. These events also arrive asynchronously, but the client may wish to explicitly wait for them by calling **XSync** after calling a function that can cause the server to generate events.
- **Errors**: Some functions return **Status**, an integer error indication. If the function fails, it returns a zero. If the function returns a status of zero, it has not updated the return arguments. Because C does not provide multiple return values, many functions must return their results by writing into clientpassed storage. By default, errors are handled either by a standard library function or by one that you provide. Functions that return pointers to strings return NULL pointers if the string does not exist.

  The X server reports **protocol errors** at the time that it detects them. If more than one error could be generated for a given request, the server can report any of them.

Because Xlib usually does not transmit requests to the server immediately (that is, it buffers them), errors can be reported much later than they actually occur. For debugging purposes, howev er, Xlib provides a mechanism for forcing synchronous behavior. When synchronization is enabled, errors are reported as they are generated.

When Xlib detects an error, it calls an error handler, which your program can provide. If you do not provide an error handler, the error is printed, and your program terminates.

- **Standard Header Files**: The following include files are part of the Xlib standard:
  - **<X11/Xlib.h>**: This is the main header file for Xlib. The majority of all Xlib symbols are declared by including this file. This file also contains the preprocessor symbol XlibSpecificationRelease. This symbol is defined to have the 6 in this release of the standard. (Release 5 of Xlib was the first release to have this symbol.)
  - **<X11/X.h>**: This file declares types and constants for the X protocol that are to be used by applications. It is included automatically from <X11/Xlib.h>, so application code should never need to reference this file directly.
  - **<X11/Xcms.h>**: This file contains symbols for much of the color management facilities. All functions, types, and symbols with the prefix ''Xcms'', plus the Color Conversion Contexts macros, are declared in this file. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xutil.h>**: This file declares various functions, types, and symbols used for inter-client communication and application utility functions, which are described in chapters 14 and 16. <X11/Xlib.h> must be included before including this file.
  - **<X11/Xresource.h>**: This file declares all functions, types, and symbols for the resource manager facilities. **<X11/Xlib.h>** must be included before including this file.
  - **<X11/Xatom.h>**: This file declares all predefined atoms, which are symbols with the prefix ''XA_''.
  - **<X11/cursorfont.h>**: This file declares the cursor symbols for the standard cursor font. All cursor symbols have the prefix ''XC_''.
  - **<X11/keysymdef.h>**: This file declares all standard KeySym values, which are symbols with the prefix ''XK_''. The KeySyms are arranged in groups, and a preprocessor symbol controls inclusion of each group. The preprocessor symbol must be defined prior to inclusion of the file to obtain the associated values. The preprocessor symbols are
    - XK_MISCELLANY
    - XK_XKB_KEYS
    - XK_3270
    - XK_LATIN1
    - XK_LATIN2
    - XK_LATIN3
    - XK_LATIN4
    - XK_KATAKANA
    - XK_ARABIC
    - XK_CYRILLIC
    - XK_GREEK

    - XK_TECHNICAL
    - XK_SPECIAL
    - XK_PUBLISHING
    - XK_APL
    - XK_HEBREW
    - XK_THAI
    - XK_KOREAN
  - **<X11/keysym.h>**: This file defines the preprocessor symbols listed above and then includes **<X11/keysymdef.h>**.
  - **<X11/Xlibint.h>**: This file declares all the functions, types, and symbols used for extensions. This file automatically includes **<X11/Xlib.h>**.
  - **<X11/Xproto.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xlibint.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/Xprotostr.h>**: This file declares types and symbols for the basic X protocol, for use in implementing extensions. It is included automatically from **<X11/Xproto.h>**, so application and extension code should never need to reference this file directly.
  - **<X11/X10.h>**: This file declares all the functions, types, and symbols used for the X10 compatibility functions.

- **KeySyms**: A KeySym is X11's symbolic meaning for a keyboard key.

  A keyboard event gives you a **KeyCode**, which is closer to "which physical/logical key was pressed." A **KeySym** is closer to "what symbol does that key currently mean?" The Xlib docs define a KeyCode as a physical or logical key code and a KeySym as the encoding of the symbol on the keycap.

  A keycode by itself is not enough, because the same physical key can mean different things depending on keyboard layout and modifiers.

- **Atoms**: An Atom is an integer ID that stands for a string name known to the X server.

  X11 uses atoms heavily for **window properties, property types, selections**, and **ClientMessage** protocols. Xlib's documentation describes a window property as named, typed data; the property name has a unique identifier called an atom, and property types are also represented using atoms.

  Instead of constantly sending strings like:

```
"WM_DELETE_WINDOW"
"_NET_WM_NAME"
"UTF8_STRING"
"WM_PROTOCOLS"
```

  clients ask the server for an integer identifier:

```
Atom wm_delete = XInternAtom(display, "WM_DELETE_WINDOW",
    ↪ False);
Atom wm_protocols = XInternAtom(display, "WM_PROTOCOLS",
    ↪ False);
```

Then they use those Atom values in later protocol requests.

- **Extensions**: An **X extension** is an addition to the core X11 protocol. The core X protocol has a fixed set of requests, events, errors, and objects. Extensions allow the server and clients to support extra functionality without changing the original core protocol.

  Examples include

```
RANDR     resizing/rotating screens, monitor layout
XRender   improved 2D rendering/compositing primitives
XInput2   advanced input devices/events
XFixes    fixes and additions to older X behavior
Composite off-screen window compositing support
Shape     non-rectangular windows
GLX       OpenGL integration with X
```

  Before using an extension, a client usually checks whether the server supports it. Xlib provides functions such as:

```
XQueryExtension(display, "RANDR", &major, &first_event,
    ↪ &first_error);
```

- **Generic values and types**: The following symbols are defined by Xlib and used throughout the manual.
  - Xlib defines the type **Bool** and the Boolean values **True** and **False**.
  - **None** is the universal null resource ID or atom.
  - The type **XID** is used for generic resource IDs.
  - The type **XPointer** is defined to be char* and is used as a generic opaque pointer to data.
- **Naming and Argument Conventions within Xlib**: Xlib follows a number of conventions for the naming and syntax of the functions. Given that you remember what information the function requires, these conventions are intended to make the syntax of the functions more predictable.

  The major naming conventions are:

  - To differentiate the X symbols from the other symbols, the library uses mixed case for external symbols. It leaves lowercase for variables and all uppercase for user macros, as per existing convention.
  - All Xlib functions begin with a capital X.

- The beginnings of all function names and symbols are capitalized.
- All user-visible data structures begin with a capital X. More generally, anything that a user might dereference begins with a capital X.
- Macros and other symbols do not begin with a capital X. To distinguish them from all user symbols, each word in the macro is capitalized.
- All elements of or variables in a data structure are in lowercase. Compound words, where needed, are constructed with underscores (_).
- The display argument, where used, is always first in the argument list.
- All resource objects, where used, occur at the beginning of the argument list immediately after the display argument.
- When a graphics context is present together with another type of resource (most commonly,adrawable), the graphics context occurs in the argument list after the other resource. Drawables outrank all other resources.
- Source arguments always precede the destination arguments in the argument list.
- The x argument always precedes the y argument in the argument list.
- The width argument always precedes the height argument in the argument list.
- Where the x, y, width, and height arguments are used together, the x and y arguments
- always precede the width and height arguments.
- Where a mask is accompanied with a structure, the mask always precedes the pointer to the structure in the argument list.

- **Programming Considerations**: The major programming considerations are:
  - Coordinates and sizes in X are actually 16-bit quantities. This decision was made to minimize the bandwidth required for a given lev el of performance. Coordinates usually are declared as an int in the interface. Values larger than 16 bits are truncated silently. Sizes (width and height) are declared as unsigned quantities.
  - Keyboards are the greatest variable between different manufacturers' workstations. If you want your program to be portable, you should be particularly conservative here.
  - Many display systems have limited amounts of off-screen memory. If you can, you should minimize use of pixmaps and backing store.
  - The user should have control of his screen real estate. Therefore, you should write your applications to react to window management rather than presume control of the entire screen. What you do inside of your top-level window, however, is up to your application. For further information, see chapter 14 and the Inter-Client Communication Conventions

- **Character Sets and Encodings**: Some of the Xlib functions make reference to specific character sets and character encodings. The following are the most common:
  - **X Portable Character Set**: A basic set of 97 characters, which are assumed to exist in all locales supported by Xlib. This set contains the following characters:

a..z A..Z 0..9
!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~
<space>, <tab>, <newline>

This set is the left/lower half of the graphic character set of ISO8859-1 plus space, tab, and newline. It is also the set of graphic characters in 7-bit ASCII plus the same three control characters. The actual encoding of these characters on the host is system dependent.

- **Host Portable Character Encoding**: The encoding of the X Portable Character Set on the host. The encoding itself is not defined by this standard, but the encoding must be the same in all locales supported by Xlib on the host. If a string is said to be in the Host Portable Character Encoding, then it only contains characters from the X Portable Character Set, in the host encoding.
  - **Latin-1**: The coded character set defined by the ISO8859-1 standard.
  - **Latin Portable Character Encoding**: The encoding of the X Portable Character Set using the Latin-1 codepoints plus ASCII control characters. If a string is said to be in the Latin Portable Character Encoding, then it only contains characters from the X Portable Character Set, not all of Latin-1.
  - **STRING Encoding**: Latin-1, plus tab and newline.
  - **POSIX Portable Filename Character Set**: The set of 65 characters, which can be used in naming files on a POSIX-compliant host, that are correctly processed in all locales. The set is:

$$a..z \quad A..Z \quad 0..9 \quad ._- .$$

- **Formatting Conventions**:
  - Global symbols are printed in this special font. These can be either function names, symbols defined in include files, or structure names. When declared and defined, function arguments are printed in italics. In the explanatory text that follows, they usually are printed in regular type.
  - Each function is introduced by a general discussion that distinguishes it from other functions. The function declaration itself follows, and each argument is specifically explained. Although ANSI C function prototype syntax is not used, Xlib header files normally declare functions using function prototypes in ANSI C environments. General discussion of the function, if any is required, follows the arguments. Where applicable, the last paragraph of the explanation lists the possible Xlib error codes that the function can generate.
  - To eliminate any ambiguity between those arguments that you pass and those that a function returns to you, the explanations for all arguments that you pass start with the word **specifies** or, in the case of multiple arguments, the word **specify**. The explanations for all arguments that are returned to you start with the word **returns** or, in the case of multiple arguments, the word **return**. The explanations for all arguments that you can pass and are returned start with the words **specifies** and **returns**.
  - Any pointer to a structure that is used to return a value is designated as such by the _return suffix as part of its name. All other pointers passed to these functions are used for reading only. A few arguments use pointers to structures that are used for both input and output and are indicated by using the _in_out suffix.

- **Clients**: A client is any program that connects to the *X* server and sends it requests.

  The X server controls the display, keyboard, mouse, screens, windows, pixmaps, graphics contexts, fonts, cursors, and other X resources. A client is a program using that server.

  So in X, the "client" is not necessarily a remote machine. It is usually just a local process on your own computer.

2. Display functions

- **Intro**: Before your program can use a display, you must establish a connection to the X server. Once you have established a connection, you then can use the Xlib macros and functions discussed in this chapter to return information about the display. We will discuss how to
  - Open (connect to) the display
  - Obtain information about the display, image formats, or screens
  - Generate a **NoOperation** protocol request
  - Free client-created data
  - Close (disconnect from) a display
  - Use X Server connection close operations
  - Use Xlib with threads
  - Use internal connections
- **Opening the Display**: To open a connection to the X server that controls a display, use **XOpenDisplay**

```
Display *XOpenDisplay(char* display_name)
```

  - **display_name** Specifies the hardware display name, which determines the display and communications domain to be used. On a POSIX-conformant system, if the display_name is NULL, it defaults to the value of the DISPLAY environment variable.

  Note: The encoding and interpretation of the display name are implementation-dependent. Strings in the Host Portable Character Encoding are supported; support for other characters is implementation-dependent. On POSIX-conformant systems, the display name or DISPLAY environment variable can be a string in the format:

  protocol/hostname:number.screen_number

  - **protocol**: Specifies a protocol family or an alias for a protocol family. Supported protocol families are implementation dependent. The protocol entry is optional. If protocol is not specified, the / separating protocol and hostname must also not be specified.
  - **hostname**: Specifies the name of the host machine on which the display is physically attached. You follow the hostname with either a single colon (:) or a double colon (::).
  - **number**: Specifies the number of the display server on that host machine. You may optionally follow this display number with a period (.). A single CPU can have more than one display. Multiple displays are usually numbered starting with zero.
  - **screen_number**: Specifies the screen to be used on that server. Multiple screens can be controlled by a single X server. The screen_number sets an internal variable that can be accessed by using the **DefaultScreen** macro or the **XDefaultScreen** function if you are using languages other than C

  For example, the following would specify screen 1 of display 0 on the machine named ‘‘dualheaded’’:

The **XOpenDisplay** function returns a **Display** structure that serves as the connection to the X server and that contains all the information about that X server. **XOpenDisplay** connects your application to the X server through TCP or DECnet communications protocols, or through some local inter-process communication protocol. If the protocol is specified as "tcp", "inet", or "inet6", or if no protocol is specified and the hostname is a host machine name and a single colon (:) separates the hostname and display number, **XOpenDisplay** connects using **TCP streams**.

If the protocol is specified as "inet", TCP over IPv4 is used. If the protocol is specified as "inet6", TCP over IPv6 is used. Otherwise, the implementation determines which IP version is used. If the hostname and protocol are both not specified, Xlib uses whatever it believes is the fastest transport. If the hostname is a host machine name and a double colon (::) separates the hostname and display number, **XOpenDisplay** connects using DECnet. A single X server can support any or all of these transport mechanisms simultaneously. A particular Xlib implementation can support many more of these transport mechanisms.

If successful, **XOpenDisplay** returns a pointer to a **Display** structure, which is defined in **<X11/Xlib.h>.** If **XOpenDisplay** does not succeed, it returns NULL. After a successful call to **XOpenDisplay,** all of the screens in the display can be used by the client. The screen number specified in the display_name argument is returned by the **DefaultScreen** macro (or the **XDefaultScreen** function). You can access elements of the **Display** and **Screen** structures only by using the information macros or functions.

- **Obtaining Information about the Display, Image Formats, or Screens**: The Xlib library provides a number of useful macros and corresponding functions that return data from the Display structure. The macros are used for C programming, and their corresponding function equivalents are for other language bindings.

  All other members of the Display structure (that is, those for which no macros are defined) are private to Xlib and must not be used. Applications must never directly modify or inspect these private members of the Display structure.

- **Planes**: In the X11 graphics model, a pixel value is not treated only as a color; it is also a group of bits. Each bit position is a plane. For example, on a drawable with depth 24, a pixel has 24 meaningful bits, so conceptually there are 24 planes.

  Note that here **depth** is the number of meaningful bits used to represent one pixel's value in a drawable.

- **Plane mask**: A plane mask chooses which bit positions of the destination pixel value are affected.

  Wherever the mask has a 1, that pixel bit plane may be changed. Wherever the mask has a 0, that pixel bit is preserved.

- **Colormaps**: A colormap is an X server object that describes how pixel values map to actual colors. In Xlib, many drawing functions do not directly use RGB colors. They use pixel values. A colormap is what lets the X server interpret those pixel values as colors.

  Pixel value → colormap → actual display color

- **Graphics contexts**: A graphics context in Xlib is an object that stores the drawing state used by graphics operations.

Instead of passing every drawing option directly to functions like XDrawLine, XFillRectangle, or XDrawString, Xlib groups many of those options into a **GC (graphics context)**.

```
XDrawLine(display, window, gc, x1, y1, x2, y2);
```

The coordinates are passed directly, but things like the line color, line width, fill style, clipping region, drawing function, font, and plane mask are taken from the gc.

The Xlib documentation describes a graphics context as holding most graphics operation attributes, including line width, line style, plane mask, foreground, background, tile, stipple, clipping region, end style, and join style. The internal contents of a GC are private to Xlib.

- **Default screen**: The default screen in Xlib is the screen number that Xlib chooses as the main screen for a given `Display*`.
- **Visuals**: A visual describes how a pixel value should be interpreted as a color on a particular screen.

  The visual describes the format/rules for interpreting pixel values, while the colormap provides the actual color mapping data when that visual class uses a lookup table.

  - **Visual**: What kind of pixel-to-color interpretation is used
  - **Colormap**: The table/mapping used by that interpretation

  The core Xlib visual classes are:

  - StaticGray
  - GrayScale
  - StaticColor
  - PseudoColor
  - TrueColor
  - DirectColor

  A `Visual*` is an Xlib object describing one supported color interpretation scheme.

- **Display Macros**: Applications should not directly modify any part of the Display and Screen structures. The members should be considered read-only, although they may change as the result of other operations on the display.

```
AllPlanes
unsigned long XAllPlanes()
```

Both return a value with all bits set to 1 suitable for use in a plane argument to a procedure.

Both BlackPixel and WhitePixel can be used in implementing a monochrome application. These pixel values are for permanently allocated entries in the default colormap. The actual RGB (red, green, and blue) values are settable on some screens and, in any case, may not actually be black or white. The names are intended to convey the expected relative intensity of the colors.

```
BlackPixel(display, screen_number)
unsigned long XBlackPixel (display, screen_number)
```

- display Specifies the connection to the X server.
- screen_number Specifies the appropriate screen number on the host server.

Both return the black pixel value for the specified screen.

```
WhitePixel (display, screen_number)
unsigned long XWhitePixel (display, screen_number)
```

Both return the white pixel value for the specified screen.

```
ConnectionNumber(display)
int XConnectionNumber(display)
```

Both return a connection number for the specified display. On a POSIX-conformant system, this is the file descriptor of the connection.

Internally, Xlib is communicating with the X server over something like a Unix-domain socket or TCP socket, and ConnectionNumber exposes that descriptor.

```
DefaultColormap(display, screen_number)
Colormap XDefaultColormap(display, screen_number)
```

Both return the default colormap ID for allocation on the specified screen. Most routine allocations of color should be made out of this colormap.

```
DefaultDepth(display, screen_number)
int XDefaultDepth(display, screen_number)
```

Both return the depth (number of planes) of the default root window for the specified screen. Other depths may also be supported on this screen (see XMatchVisualInfo).

```
int* XListDepths(display, screen_number, count_return)
```

– count_return returns the number of depths

The **XListDepths** function returns the array of depths that are available on the specified screen. If the specified screen_number is valid and sufficient memory for the array can be allocated, **XListDepths** sets count_return to the number of available depths. Otherwise, it does not set count_return and returns NULL. To release the memory allocated for the array of depths, use **XFree.**

XListDepths ask the *X* server:

"*For this screen, what drawable/window depths are supported*"

```
DefaultGC(display, screen_number)
GC XDefaultGC(display, screen_number)
```

Both return the default graphics context for the root window of the specified screen. This GC is created for the convenience of simple applications and contains the default GC components with the foreground and background pixel values initialized to the black and white pixels for the screen, respectively. You can modify its contents freely because it is not used in any Xlib function. This GC should never be freed.

```
DefaultRootWindow(display)
Window XDefaultRootWindow(display)
```

Both return the root window for the default screen.

```
DefaultScreenOfDisplay(display)
Screen* XDefaultScreenOfDisplay(display)
```

Both return a pointer to the default screen.

```
ScreenOfDisplay(display, screen_number)
Screen* XScreenOfDisplay(display, screen_number)
```

Both return a pointer to the indicated screen

```
DefaultScreen(display)
int XDefaultScreen(display)
```

Both return the default screen number referenced by the **XOpenDisplay** function. This macro or function should be used to retrieve the screen number in applications that will use only a single screen.

```
DefaultVisual(display, screen_number)
Visual* XDefaultVisual(display, screen_number)
```

Both return the default visual type for the specified screen.

```
DisplayCells(display, screen_number)
int XDisplayCells(display, screen_number)
```

Both return the number of entries in the default colormap.

```
DisplayPlanes(display, screen_number)
int XDisplayPlanes(display, screen_number)
```

Both return the depth of the root window of the specified screen.

```
DisplayString(display)
char* XDisplayString(display)
```

Both return the string that was passed to XOpenDisplay when the current display was opened. On POSIX-conformant systems, if the passed string was NULL, these return the value of the DIS- PLAY environment variable when the current display was opened. These are useful to applications that invoke the fork system call and want to open a new connection to the same display from the child process as well as for printing error messages.

```
long XExtendedMaxRequestSize(display)
```

The **XExtendedMaxRequestSize** function returns zero if the specified display does not support an extended-length protocol encoding; otherwise, it returns the maximum request size (in 4-byte units) supported by the server using the extended-length encoding. The Xlib functions XDraw- Lines, **XDrawArcs, XFillPolygon, XChange-Property, XSetClipRectangles,** and **XSetRegion** will use the extended-length encoding as necessary, if supported by the server. Use of the extended-length encoding in other Xlib functions (for example, **XDrawPoints, XDrawRectangles**, **XDrawSegments, XFillArcs, XFillRectangles, XPutImage)** is permitted but not required; an Xlib implementation may choose to split the data across multiple smaller requests instead.

```
long XMaxRequestSize(display)
```

The XMaxRequestSize function returns the maximum request size (in 4-byte units) supported by the server without using an extended-length protocol encoding. Single protocol requests to the server can be no larger than this size unless an extended-length protocol encoding is supported by the server. The protocol guarantees the size to be no smaller than 4096 units (16384 bytes). Xlib automatically breaks data up into multiple protocol requests as necessary for the following functions: **XDrawPoints, XDrawRectangles, XDrawSegments, XFillArcs, XFillRectangles,** and **XPutImage**

```
LastKnownRequestProcessed(display)
unsigned long XLastKnownRequestProcessed(display)
```

Both extract the full serial number of the last request known by Xlib to have been processed by the X server. Xlib automatically sets this number when replies, events, and errors are received.

```
NextRequest (display)
unsigned long XNextRequest (display)
```

Both extract the full serial number that is to be used for the next request. Serial numbers are maintained separately for each display connection.

Note: In Xlib, the **serial number** of a request is the sequence number assigned to each protocol request sent from your client to the X server.

```
ProtocolVersion(display)
int XProtocolVersion(display)
```

Both return the major version number (11) of the X protocol associated with the connected dis play.

```
ProtocolRevision (display)
int XProtocolRevision (display)
```

Both return the minor protocol revision number of the X server.

```
QLength (display)
int XQLength(display)
```

Both return the length of the event queue for the connected display. Note that there may be more events that have not been read into the queue yet (see **XEventsQueued).**

```
RootWindow(display, screen_number)
Window XRootWindow(display, screen_number)
```

Both return the root window. These are useful with functions that need a drawable of a particular screen and for creating top-level windows.

```
ScreenCount(display)
int XScreenCount(display)
```

Both return the number of available screens.

```
ServerVendor (display)
char* XServerVendor (display)
```

Both return a pointer to a null-terminated string that provides some identification of the owner of the X server implementation. If the data returned by the server is in the Latin Portable Character Encoding, then the string is in the Host Portable Character Encoding. Otherwise, the contents of the string are implementation-dependent.

```
VendorRelease (display)
int XVendorRelease (display)
```

Both return a number related to a vendor's release of the X server.

- **Image data formats**: In Xlib, **XY format** and **Z format** are two different memory layouts for image data. They mainly matter when using functions like:

```
XGetImage(...)
XCreateImage(...)
XPutImage(...)
```

and constants such as

```
XYBitmap
XYPixmap
ZPixmap
```

  - **Z format (ZPixmap)**: ZPixmap stores pixels pixel-by-pixel. Each pixel value contains all of its plane bits together. For a 24-bit image, one pixel might contain something like:

RRRRRRRR GGGGGGGG BBBBBBBB

So the image is stored as a normal packed array of pixel values. This is usually the easiest format to understand and use. If you want to read or write individual pixels with **XGetPixel / XPutPixel,** ZPixmap is the common choice.

- **XY format (XYPixmap)**: Instead of storing whole pixels together, it stores one bit-plane at a time. In a 4-bit-deep pixmap, each pixel has 4 bits. In XYPixmap, the image data is stored as separate planes.

  plane 0: bit 0 of every pixel
  plane 1: bit 1 of every pixel
  plane 2: bit 2 of every pixel
  plane 3: bit 3 of every pixel

  So,

  plane 0: pixel0_bit0 pixel1_bit0 pixel2_bit0 ...
  plane 1: pixel0_bit1 pixel1_bit1 pixel2_bit1 ...
  plane 2: pixel0_bit2 pixel1_bit2 pixel2_bit2 ...
  plane 3: pixel0_bit3 pixel1_bit3 pixel2_bit3 ...

  This is why it is called XY format: it stores bitmap-style x/y image planes, one plane after another.

- **XY format (XYBitmap)**: XYBitmap is a special case of XY format for depth 1 images. A bitmap has only one bit per pixel. So, there is only one plane. This is used for masks, stipples, cursors, icons, and other monochrome data.

- **Scanlines**: A scanline is one horizontal row of pixels in an image. If an image is

  $$\begin{aligned} \text{width} &= 640 \\ \text{height} &= 480 \end{aligned}$$

  then it has 480 scanlines, each containing 640 pixels.

  In Xlib, the scanline unit is the size, in bits, of the storage chunks used to represent a scanline, especially for bitmap/plane-style image data.

  Suppose you have a 1-bit bitmap image

  pixel: 0 1 2 3 4 5 6 7 8 9 ...
  value: 1 0 1 1 0 0 1 0 1 1 ...

  The pixels are individual bits, but memory is addressed in bytes or words, not individual bits. So X needs to know how the bits are grouped.

  If the scanline unit is 8, the scanline is grouped like this:

  bits 0-7 | bits 8-15 | bits 16-23 | ...

  That grouping affects how X interprets the bits in memory.

  If the image width is not a multiple of the scanline unit/padding requirement, the scanline is padded at the end. The extra bits are not pixels; they are just unused storage so that the next scanline starts at the required boundary.

- **Image format functions and macros**: Applications are required to present data to the X server in a format that the server demands. To help simplify applications, most of the work required to convert the data is provided by Xlib

The **XPixmapFormatValues** structure provides an interface to the pixmap format information that is returned at the time of a connection setup. It contains:

```
typedef struct {
    int depth;
    int bits_per_pixel;
    int scanline_pad;
} XPixmapFormatValues;
```

To obtain the pixmap format information for a given display, use **XListPixmapFormats**.

```
XPixmapFormatValues* XListPixmapFormats (display,
↪  count_return)
```

The **XListPixmapFormats** function returns an array of **XPixmapFormatValues** structures that describe the types of Z format images supported by the specified display. If insufficient memory is available, **XListPixmapFormats** returns NULL. To free the allocated storage for the **XPixmapFormatValues** structures, use **XFree.**

```
ImageByteOrder(display)
int XImageByteOrder(display)
```

Both specify the required byte order for images for each scanline unit in XY format (bitmap) or for each pixel value in Z format. The macro or function can return either **LSBFirst** or **MSBFirst.**

```
BitmapUnit (display)
int XBitmapUnit(display)
```

Both return the size of a bitmap's scanline unit in bits. The scanline is calculated in multiples of this value.

```
BitmapBitOrder (display)
int XBitmapBitOrder(display)
```

Within each bitmap unit, the left-most bit in the bitmap as displayed on the screen is either the least signifiant or most signifiant bit in the unit. This macro or function can return **LSBFirst** or **MSBFirst.**

```
BitmapPad (display)
int XBitmapPad (display)
```

Each scanline must be padded to a multiple of bits returned by this macro or function.

```
DisplayHeight(display, screen_number)
int XDisplayHeight(display, screen_number)
```

Both return an integer that describes the height of the screen in pixels.

```
DisplayHeightMM (display, screen_number)
int XDisplayHeightMM(display, screen_number)
```

Both return the height of the specified screen in millimeters.

```
DisplayWidth (display, screen_number)
int XDisplayWidth (display, screen_number)
```

Both return the width of the screen in pixels.

```
DisplayWidthMM (display, screen_number)
int XDisplayWidthMM (display, screen_number)
```

Both return the width of the specified screen in millimeters.

- **Backing stores**: A backing store is an optional server-side saved copy of a window's contents.

  Normally, if part of your window is covered and then uncovered, the X server may not remember what was there. Instead, it sends your program an Expose event telling you:

  "*this part of your window became visible again; redraw it.*"

  With backing store, the server may keep a copy of the obscured window contents. Then, when the window becomes visible again, the server can restore the pixels itself without requiring your client to redraw.

  Important: backing store is only a hint. Your program must still handle Expose events correctly. The X server is allowed not to preserve the contents, especially if it decides the backing store would be too expensive.

- **Save-unders**: A save-under is also a server-side pixel preservation hint, but it is used for a different purpose.

  Backing store preserves the contents of the window itself.

  Save-under preserves the contents of whatever is underneath a temporary window.

This is mainly for things like:

- menus
- tooltips
- popup windows
- drag boxes
- temporary overlays

Without save-under:

menu appears
↓
menu covers part of another window
↓
menu disappears
↓
the window underneath may need Expose events and redraws

With save-under:

menu appears
↓
server saves pixels underneath the menu
↓
menu disappears
↓
server restores the pixels underneath

- **Event masks**: An event mask tells the X server which events your client wants to receive for a window. X is event-driven. Your program usually waits for events like:
  - key press
  - mouse click
  - mouse movement
  - window exposure
  - resize
  - map/unmap
  - destroy
  - focus changes
  - property changes

  But the server does not automatically send every possible event to every client. You select the events you care about using an event mask.

  For example,

```
XSelectInput(
    display,
    window,
    ExposureMask  KeyPressMask  ButtonPressMask |
    ↪  StructureNotifyMask
);
```

If you want to redraw your window when it becomes visible or damaged, you must select:

```
ExposureMask
```

Then the server can send you **Expose** events.

- **Screen information macros**: The following lists the C language macros, their corresponding function equivalents that are for other language bindings, and what data they both can return. These macros or functions all take a pointer to the appropriate screen structure.

```
BlackPixelOfScreen (screen)
unsigned long XBlackPixelOfScreen (screen)
```

Both return the black pixel value of the specified screen.

```
WhitePixelOfScreen (screen)
unsigned long XWhitePixelOfScreen (screen)
```

Both return the white pixel value of the specified screen.

```
CellsOfScreen (screen)
int XCellsOfScreen(screen)
```

Both return the number of colormap cells in the default colormap of the specified screen.

```
DefaultColormapOfScreen (screen)
Colormap XDefaultColormapOfScreen (screen)
```

  - **Screen* screen**: screen Specifies the appropriate Screen structure.

Both return the default colormap of the specified screen.

```
DefaultDepthOfScreen (screen)
int XDefaultDepthOfScreen (screen)
```

Both return the depth of the root window.

```
DefaultGCOfScreen (screen)
GC XDefaultGCOfScreen (screen)
```

Both return a default graphics context (GC) of the specified screen, which has the same depth as the root window of the screen. The GC must never be freed.

```
DefaultVisualOfScreen (screen)
Visual *XDefaultVisualOfScreen (screen)
```

Both return the default visual of the specified screen. For information on visual types

```
DoesBackingStore (screen)
int XDoesBackingStore(screen)
```

Both return a value indicating whether the screen supports backing stores. The value returned can be one of **WhenMapped, NotUseful,** or Always

```
DoesSaveUnders (screen)
Bool XDoesSaveUnders (screen)
```

Both return a Boolean value indicating whether the screen supports save unders. If **True,** the screen supports save unders. If **False,** the screen does not support save unders (see section 3.2.5).

```
DisplayOfScreen (screen)
Display *XDisplayOfScreen(screen)
```

Both return the display of the specified screen.

```
int XScreenNumberOfScreen(screen)
```

The **XScreenNumberOfScreen** function returns the screen index number of the specified screen.

```
EventMaskOfScreen (screen)
long XEventMaskOfScreen (screen)
```

Both return the event mask of the root window for the specified screen at connection setup time.

```
WidthOfScreen (screen)
int XWidthOfScreen (screen)
```

Both return the width of the specified screen in pixels.

```
HeightOfScreen (screen)
int XHeightOfScreen(screen)
```

Both return the height of the specified screen in pixels.

```
WidthMMOfScreen (screen)
int XWidthMMOfScreen (screen)
```

Both return the width of the specified screen in millimeters.

```
HeightMMOfScreen (screen)
int XHeightMMOfScreen(screen)
```

Both return the height of the specified screen in millimeters.

```
MaxCmapsOfScreen (screen)
int XMaxCmapsOfScreen(screen)
```

Both return the maximum number of installed colormaps supported by the specified screen

```
MinCmapsOfScreen (screen)
int XMinCmapsOfScreen(screen)
```

Both return the minimum number of installed colormaps supported by the specified screen

```
PlanesOfScreen (screen)
int XPlanesOfScreen(screen)
```

Both return the depth of the root window.

```
RootWindowOfScreen (screen)
Window XRootWindowOfScreen (screen)
```

Both return the root window of the specified screen.

- **Generating a NoOperation Protocol Request**: To execute a **NoOperation** protocol request, use **XNoOp**

```
XNoOp(display)
```

The **XNoOp** function sends a **NoOperation** protocol request to the X server, thereby exercising the connection.

- **Freeing client-created data**: To free in-memory data that was created by an Xlib function, use **XFree**.

```
XFree(void* data)
```

The **XFree** function is a general-purpose Xlib routine that frees the specified data. You must use it to free any objects that were allocated by Xlib, unless an alternate function is explicitly specified for the object. A NULL pointer cannot be passed to this function.

- **Closing the Display**: To close a display or disconnect from the X server, use **XCloseDisplay**.

```
XCloseDisplay(Display* display)
```

The **XCloseDisplay** function closes the connection to the X server for the display specified in the **Display** structure and destroys all windows, resource IDs (**Window, Font, Pixmap, Colormap, Cursor,** and **GContext),** or other resources that the client has created on this display, unless the close-down mode of the resource has been changed (see **XSetCloseDownMode).** Therefore, these windows, resource IDs, and other resources should never be referenced again or an error will be generated. Before exiting, you should call **XCloseDisplay** explicitly so that any pending errors are reported as **XCloseDisplay** performs a final **XSync** operation.

XCloseDisplay can generate a **BadGC** error.

Xlib provides a function to permit the resources owned by a client to survive after the client's connection is closed. To change a client's close-down mode, use **XSetCloseDownMode**

```
XSetCloseDownMode (Display* display, int close_mode)
```

  - **close_mode**: close_mode Specifies the client close-down mode. You can pass **DestroyAll, RetainPermanent,** or **RetainTemporary.**

The **XSetCloseDownMode** defines what will happen to the client's resources at connection close. A connection starts in **DestroyAll** mode.

XSetCloseDownMode can generate a **BadValue** error.

- **Selections**: A selection is the X Window System's mechanism for clipboard-like data transfer.

  The important point is that the selected data is usually not stored in the X server. Instead, one client owns a selection, and another client requests that data from the owner.

  Common selections are

```
PRIMARY
SECONDARY
CLIPBOARD
```

  - **PRIMARY** usually the currently highlighted text
  - **CLIPBOARD** usually explicit copy/paste, like Ctrl+C / Ctrl+V
  - **SECONDARY** older, rarely used

  The selection itself is named by an Atom, for example

```
Atom clipboard = XInternAtom(display, "CLIPBOARD", False);
```

  A client becomes the owner of a selection with

```
XSetSelectionOwner(display, selection_atom, owner_window,
↪  timestamp);
```

  For example,

```
XSetSelectionOwner(display, clipboard, window, CurrentTime)
```

  You can ask who owns a selection with:

```
Window owner = XGetSelectionOwner(display, clipboard)
```

  Another client requests the selection using:

```
XConvertSelection(
    display,
    selection,
    target,
    property,
    requestor,
    time
);
```

This asks the selection owner:

"*Please convert your selection data to this target type and store it on this property of my window.*"

So the flow is

client A owns CLIPBOARD
↓
client B calls XConvertSelection
↓
client A receives SelectionRequest
↓
client A writes data to a property on client B's window
↓
client A sends SelectionNotify
↓
client B reads the property
.

This is why selections are more like negotiated inter-client data transfer than a simple global clipboard buffer.

The main events are

```
SelectionClear
SelectionRequest
SelectionNotify
```

  - **SelectionClear**: You lost ownership of the selection.
  - **SelectionRequest**: Another client is asking you for the selection data.
  - **SelectionNotify**: The selection conversion request completed.

A correct clipboard owner must handle **SelectionRequest.** If it exits, the selected data may disappear unless a clipboard manager has saved it.

- **Grabs**: A grab gives one client temporary exclusive control over some input.

There are two major kinds:

1. Pointer grabs
2. Keyboard grabs

A grab can be **active** or **passive**.

- **Active**: Happens immediately when you request it
- **Passive**: A passive grab does not take effect immediately. Instead, it activates when some input pattern occurs

For the pointer:

```
int XGrabPointer(
    Display* display,
    Window grab_window,
    bool owner_events,
    unsigned int event_mask,
    int pointer_mode,
    int keyboard_mode,
    Window confine_to,
    Cursor cursor,
    Time time
);
```

- display Specifies the connection to the X server.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.
- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync.**
- confine_to Specifies the window to confine the pointer in or None.
- cursor Specifies the cursor that is to be displayed during the grab or None.
- time Specifies the time. You can pass either a timestamp or CurrentTime

The XGrabPointer() function actively grabs control of the pointer and returns GrabSuccess if the grab was successful. Further pointer events are reported only to the grabbing client. XGrabPointer() can generate **BadCursor, BadValue,** and **BadWindow** errors.

For the keyboard:

```
int XGrabKeyboard(
    Display* display,
    Window grab_window,
    Bool owner_events,
    int pointer_mode,
    int keyboard_mode,
    Time time
);
```

- display Specifies the connection to the X server.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass GrabModeSync or GrabModeAsync.
- keyboard_mode Specifies further processing of keyboard events. You can pass GrabModeSync or GrabModeAsync.
- time Specifies the time. You can pass either a timestamp or CurrentTime.

The **XGrabKeyboard()** function actively grabs control of the keyboard and generates **FocusIn** and **FocusOut** events. Further key events are reported only to the grabbing client.

Once a client has an active pointer grab, pointer events are directed according to the grab rules rather than normal window picking. Typical uses are

- menus
- drag-and-drop
- resizing a window
- moving a window
- modal interactions
- screen lockers

For example, while dragging a scrollbar, you do not want pointer events to suddenly go to another window just because the pointer moved outside the scrollbar. A pointer grab lets the scrollbar continue receiving motion/release events until the drag finishes.

You release grabs with:

```
XUngrabPointer(display, time)
XUngrabKeyboard(display, time)
```

A passive grab does not take effect immediately. Instead, it activates when some input pattern occurs.

For keyboard shortcuts:

```
int XGrabKey(
    Display* display,
    int keycode,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    int pointer_mode,
    int keyboard_mode
);
```

- display Specifies the connection to the X server.
- keycode Specifies the KeyCode or **AnyKey.**
- modifiers Specifies the set of keymasks or **AnyModifier.** The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the keyboard events are to be reported as usual.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync**

The **XGrabKey()** function establishes a passive grab on the keyboard. **XGrabKey()** can generate **BadAccess** , **BadValue** , and **BadWindow** errors.

For mouse buttons:

```
int XGrabButton(
    Display* display,
    unsigned int button,
    unsigned int modifiers,
    Window grab_window,
    Bool owner_events,
    unsigned int event_mask,
    int pointer_mode,
    int keyboard_mode,
    Window confine_to,
    Cursor cursor
);
```

- display Specifies the connection to the X server.
- button Specifies the pointer button that is to be grabbed or **AnyButton.**
- modifiers Specifies the set of keymasks or **AnyModifier.** The mask is the bitwise inclusive OR of the valid keymask bits.
- grab_window Specifies the grab window.
- owner_events Specifies a Boolean value that indicates whether the pointer events are to be reported as usual or reported with respect to the grab window if selected by the event mask.

- event_mask Specifies which pointer events are reported to the client. The mask is the bitwise inclusive OR of the valid pointer event mask bits.
- pointer_mode Specifies further processing of pointer events. You can pass **GrabModeSync** or **GrabModeAsync.**
- keyboard_mode Specifies further processing of keyboard events. You can pass **GrabModeSync** or **GrabModeAsync.**
- confine_to Specifies the window to confine the pointer in or None.
- cursor Specifies the cursor that is to be displayed or None.

The XGrabButton() function establishes a passive grab. **XGrabButton()** can generate **BadCursor, BadValue,** and **BadWindow** errors.

Xlib defines constants for the standard mouse buttons:

- **Button1**: Typically the left mouse button.
- **Button2**: Typically the middle mouse button (or scroll wheel click).
- **Button3**: Typically the right mouse button.
- **Button4 / Button5**: Often assigned to the scroll wheel up/down actions.
- **AnyButton**: A special constant used in functions like XGrabButton to indicate that the operation should apply to any mouse button

This is how a window manager can say:

"*When Mod1 + Button1 is pressed over this window, give me control.*"

The grab becomes active only when the specified key/button/modifier combination occurs.

- **Keymask modifiers**: In Xlib, keymask modifiers are bitwise constants used to describe the state of modifier keys (like Shift, Control, or Alt) during an input event. These masks are commonly found in the state member of event structures such as XKeyEvent or used as arguments in functions like **XGrabKey.**
  - **ShiftMask**: The Shift key is pressed.
  - **LockMask**: The Caps Lock key is active.
  - **ControlMask**: The Control key is pressed.
  - **Mod1Mask**: Usually mapped to Alt or Meta.
  - **Mod2Mask**: Often mapped to Num Lock.
  - **Mod3Mask**: Sometimes mapped to AltGr.
  - **Mod4Mask**: Commonly mapped to the Super/Windows key.
  - **Mod5Mask**: Often mapped to Scroll Lock or other custom modifiers.
- **Synchronous vs asynchronous grabs**: Grabs can freeze or not freeze event processing. The modes are usually:

```
0 GrabModeAsync
1 GrabModeSync
```

With GrabModeAsync, events continue normally. With GrabModeSync, the server may freeze pointer or keyboard event processing until the grabbing client explicitly allows it to continue with:

```
int XAllowEvents(
    Display* display,
    int mode,
    Time time
);
```

- display Specifies the connection to the X server.
- event_mode Specifies the event mode. You can pass **AsyncPointer, SyncPointer, AsyncKeyboard, SyncKeyboard, ReplayPointer, ReplayKeyboard, AsyncBoth,** or **SyncBoth.**
- time Specifies the time. You can pass either a timestamp or CurrentTime.

The XAllowEvents() function releases some queued events if the client has caused a device to freeze. It has no effect if the specified time is earlier than the last-grab time of the most recent active grab for the client or if the specified time is later than the current X server time. Depending on the event_mode argument, the following occurs:

- **AsyncPointer**: If the pointer is frozen by the client, pointer event processing continues as usual. If the pointer is frozen twice by the client on behalf of two separate grabs, AsyncPointer thaws for both. AsyncPointer has no effect if the pointer is not frozen by the client, but the pointer need not be grabbed by the client.
- **SyncPointer**: If the pointer is frozen and actively grabbed by the client, pointer event processing continues as usual until the next ButtonPress or ButtonRelease event is reported to the client. At this time, the pointer again appears to freeze. However, if the reported event causes the pointer grab to be released, the pointer does not freeze. SyncPointer has no effect if the pointer is not frozen by the client or if the pointer is not grabbed by the client.
- **ReplayPointer**: If the pointer is actively grabbed by the client and is frozen as the result of an event having been sent to the client (either from the activation of a XGrabButton() or from a previous XAllowEvents() with mode SyncPointer but not from a XGrabPointer()), the pointer grab is released and that event is completely reprocessed. This time, however, the function ignores any passive grabs at or above (towards the root of) the grab_window of the grab just released. The request has no effect if the pointer is not grabbed by the client or if the pointer is not frozen as the result of an event.
- **AsyncKeyboard**: If the keyboard is frozen by the client, keyboard event processing continues as usual. If the keyboard is frozen twice by the client on behalf of two separate grabs, AsyncKeyboard thaws for both. AsyncKeyboard has no effect if the keyboard is not frozen by the client, but the keyboard need not be grabbed by the client.
- **SyncKeyboard**: If the keyboard is frozen and actively grabbed by the client, keyboard event processing continues as usual until the next KeyPress or KeyRelease event is reported to the client. At this time, the keyboard again appears to freeze. However, if the reported event causes the keyboard grab to be released, the keyboard does not freeze. SyncKeyboard has no effect if the keyboard is not frozen by the client or if the keyboard is not grabbed by the client.

- **ReplayKeyboard**: If the keyboard is actively grabbed by the client and is frozen as the result of an event having been sent to the client (either from the activation of a XGrabKey() or from a previous XAllowEvents() with mode SyncKeyboard but not from a XGrabKeyboard()), the keyboard grab is released and that event is completely reprocessed. This time, however, the function ignores any passive grabs at or above (towards the root of) the grab_window of the grab just released. The request has no effect if the keyboard is not grabbed by the client or if the keyboard is not frozen as the result of an event.
- **SyncBoth**: If both pointer and keyboard are frozen by the client, event processing for both devices continues as usual until the next ButtonPress, ButtonRelease, KeyPress, or KeyRelease event is reported to the client for a grabbed device (button event for the pointer, key event for the keyboard), at which time the devices again appear to freeze. However, if the reported event causes the grab to be released, then the devices do not freeze (but if the other device is still grabbed, then a subsequent event for it will still cause both devices to freeze). SyncBoth has no effect unless both pointer and keyboard are frozen by the client. If the pointer or keyboard is frozen twice by the client on behalf of two separate grabs, SyncBoth thaws for both (but a subsequent freeze for SyncBoth will only freeze each device once).
- **AsyncBoth**: If the pointer and the keyboard are frozen by the client, event processing for both devices continues as usual. If a device is frozen twice by the client on behalf of two separate grabs, AsyncBoth thaws for both. AsyncBoth has no effect unless both pointer and keyboard are frozen by the client.

This is useful when a client wants to inspect an event before deciding whether it should be replayed or swallowed.

XAllowEvents() can generate a **BadValue** error.

- **Grab result codes**: A grab can fail, for example

```
int status = XGrabPointer(...);
```

Possible result values include:

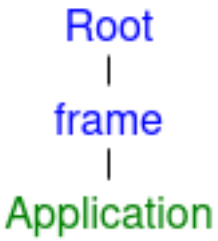
```
GrabSuccess
AlreadyGrabbed
GrabInvalidTime
GrabNotViewable
GrabFrozen
```

So a grab is not guaranteed.

- **Save-set**: A save-set is a mechanism used mainly by window managers to avoid destroying or orphaning client windows if the window manager exits. This matters because window managers often reparent client windows.

Normally, an application creates a top-level window as a child of the root window:

A reparenting window manager inserts a frame window around it:



The application window is now a child of a window owned by the window manager. Now, we have a problem...

If the window manager dies, what happens to the application windows inside its frame windows?

Without protection, those client windows could be destroyed or left in a bad state when the frame windows disappear. So the window manager adds application windows to its save-set:

```
XAddToSaveSet(display, client_window)
```

If the window manager exits or its connection to the X server closes, the server automatically reparents those save-set windows, usually back to a safe ancestor such as the root, and maps them if appropriate.

You can remove a window from the save-set with

```
XRemoveFromSaveSet(display, client_window);
```

- **Superior and inferior**: A window's **inferiors** are its descendants. A window's **superiors** are its ancestors.
- **Using X Server Connection Close Operations**: When the X server's connection to a client is closed either by an explicit call to **XCloseDisplay** or by a process that exits, the X server performs the following automatic operations:
  - It disowns all selections owned by the client (see **XSetSelectionOwner).**

- It performs an **XUngrabPointer** and **XUngrabKeyboard** if the client has actively grabbed the pointer or the keyboard.
- It performs an **XUngrabServer** if the client has grabbed the server.
- It releases all passive grabs made by the client.
- It marks all resources (including colormap entries) allocated by the client either as perma- nent or temporary, depending on whether the close-down mode is **RetainPermanent** or **RetainTemporary.** However, this does not prevent other client applications from explicitly destroying the resources (see **XSetCloseDownMode).**

When the close-down mode is **DestroyAll,** the X server destroys all of a client's resources as fol- lows:

- It examines each window in the client's save-set to determine if it is an inferior (subwin- dow) of a window created by the client. (The save-set is a list of other clients' windows that are referred to as save-set windows.) If so, the X server reparents the save-set window to the closest ancestor so that the save-set window is not an inferior of a window created by the client. The reparenting leaves unchanged the absolute coordinates (with respect to the root window) of the upper-left outer corner of the save-set window.
- It performs a **MapWindow** request on the save-set window if the save-set window is unmapped. The X server does this even if the save-set window was not an inferior of a window created by the client.
- It destroys all windows created by the client.
- It performs the appropriate free request on each nonwindow resource created by the client in the server (for example, **Font, Pixmap, Cursor, Colormap,** and **GContext).**
- It frees all colors and colormap entries allocated by a client application

Additional processing occurs when the last connection to the X server closes. An X server goes through a cycle of having no connections and having some connections. When the last connection to the X server closes as a result of a connection closing with the close_mode of **DestroyAll,** the X server does the following:

- It resets its state as if it had just been started. The X server begins by destroying all linger- ing resources from clients that have terminated in **RetainPermanent** or **RetainTemporary** mode.
- It deletes all but the predefined atom identifiers.
- It deletes all properties on all root windows (see section 4.3).
- It resets all device maps and attributes (for example, key click, bell volume, and acceleration) as well as the access control list.
- It restores the standard root tiles and cursors.
- It restores the default font path.
- It restores the input focus to state **PointerRoot**

However, the X server does not reset if you close a connection with a close-down mode set to **RetainPermanent** or **RetainTemporary**

- **Locking a display**: To lock a display in Xlib means take a mutex-like lock on the `Display*` so only one thread in your process uses that X connection at a time.

```
XLockDisplay(display)
```

While the display is locked, other threads in the same process that try to use the same Display * will block until it is unlocked.

```
XUnlockDisplay(display)
```

- **Using Xlib with threads**: To initialize support for concurrent threads, use **XInitThreads**

```
Status XInitThreads();
```

The **XInitThreads** function initializes Xlib support for concurrent threads. This function must be the first Xlib function a multi-threaded program calls, and it must complete before any other Xlib call is made. This function returns a nonzero status if initialization was successful; otherwise, it returns zero. On systems that do not support threads, this function always returns zero.

It is only necessary to call this function if multiple threads might use Xlib concurrently. If all calls to Xlib functions are protected by some other access mechanism (for example, a mutual exclusion lock in a toolkit or through explicit client programming), Xlib thread initialization is not required. It is recommended that single-threaded programs not call this function.

To lock a display across several Xlib calls, use **XLockDisplay**

```
void XLockDisplay(display)
```

The **XLockDisplay** function locks out all other threads from using the specified display. Other threads attempting to use the display will block until the display is unlocked by this thread. Nested calls to **XLockDisplay** work correctly; the display will not actually be unlocked until **XUnlockDisplay** has been called the same number of times as **XLockDisplay.** This function has no effect unless Xlib was successfully initialized for threads using **XInitThreads.**

To unlock a display, use XUnlockDisplay

```
void XUnlockDisplay(display)
```

The **XUnlockDisplay** function allows other threads to use the specified display again. Any threads that have blocked on the display are allowed to continue. Nested locking works correctly; if **XLockDisplay** has been called multiple times by a thread, then **XUnlockDisplay** must be called an equal number of times before the display is actually unlocked. This function has no effect unless Xlib was successfully initialized for threads using **XInitThreads.**

- **Using Internal Connections**: In addition to the connection to the X server, an Xlib implementation may require connections to other kinds of servers. Toolkits and clients that use multiple displays, or that use displays in combination with other inputs, need to obtain these additional connections to correctly block until input is available and need to process that input when it is available. Simple clients that use a single display and block for input in an Xlib event function do not need to use these facilities.

  To track internal connections for a display, use **XAddConnectionWatch.**

```
typedef void (*XConnectionWatchProc) (Display* display,
    ↪ XPointer client_data, int fd, Bool opening, XPointer
    ↪ watch_data)

Status XAddConnectionWatch (Display* display,
    ↪ XConnectionWatchProc procedure, XPointer client_data)
```

  – procedure Specifies the procedure to be called.
  – client_data Specifies the additional client data.

  The **XAddConnectionWatch** function registers a procedure to be called each time Xlib opens or closes an internal connection for the specified display. The procedure is passed the display, the specified client_data, the file descriptor for the connection, a Boolean indicating whether the connection is being opened or closed, and a pointer to a location for private watch data. If opening is **True,** the procedure can store a pointer to private data in the location pointed to by watch_data; when the procedure is later called for this same connection and opening is **False**, the location pointed to by watch_data will hold this same private data pointer.

  This function can be called at any time after a display is opened. If internal connections already exist, the registered procedure will immediately be called for each of them, before **XAddConnectionWatch** returns. **XAddConnectionWatch** returns a nonzero status if the procedure is successfully registered; otherwise, it returns zero

  The registered procedure should not call any Xlib functions. If the procedure directly or indirectly causes the state of internal connections or watch procedures to change, the result is not defined. If Xlib has been initialized for threads, the procedure is called with the display locked and the result of a call by the procedure to any Xlib function that locks the display is not defined unless the executing thread has externally locked the display using **XLockDisplay**

  To stop tracking internal connections for a display, use **XRemoveConnectionWatch.**

```
Status XRemoveConnectionWatch (display, procedure,
    ↪ client_data)
```

The **XRemoveConnectionWatch** function removes a previously registered connection watch procedure. The client_data must match the client_data used when the procedure was initially reg- istered.

To process input on an internal connection, use **XProcessInternalConnection**

```
void XProcessInternalConnection(Display* display, int fd)
```

The **XProcessInternalConnection** function processes input available on an internal connection. This function should be called for an internal connection only after an operating system facility (for example, select or poll) has indicated that input is available; otherwise, the effect is not defined.

To obtain all of the current internal connections for a display, use **XInternalConnectionNumbers.**

```
Status XInternalConnectionNumbers(Display* display, int**
↪  fd_return, int* count_return)
```

The **XInternalConnectionNumbers** function returns a list of the file descriptors for all internal connections currently open for the specified display. When the allocated list is no longer needed, free it by using **XFree.** This functions returns a nonzero status if the list is successfully allocated; otherwise, it returns zero.

3. Window functions

- **Intro**: In the *X* Window System, a window is a rectangular area on the screen that lets you view graphic output. Client applications can display overlapping and nested windows on one or more screens that are driven by *X* servers on one or more machines. Clients who want to create windows must first connect their program to the *X* server by calling **XOpenDisplay.** This chapter begins with a discussion of visual types and window attributes. The chapter continues with a discussion of the Xlib functions you can use to:
  - Create windows
  - Destroy windows
  - Map windows
  - Unmap windows
  - Configure windows
  - Change window stacking order
  - Change window attributes
- **ICCCM / EWMH conventions**:
  - **ICCCM (Inter-client communication conventions)**: The Inter-Client Communication Conventions Manual (ICCCM) is a foundational standard protocol for the X Window System. It establishes the rules that graphical applications (clients) must follow to cooperate, share resources (like cut and paste selections), and interact smoothly with window managers.

    The ICCCM focuses on a few core areas of client behavior to ensure a stable and predictable desktop experience:
    * **Selections and Cut Buffers**: Defines how clients share data across the global X server, enabling universal copy and paste operations between different applications.
    * **Window Manager Communication**: Establishes the rules for how applications request window sizes, positions, and decorations, ensuring that a window manager can successfully control top-level windows without relying on private protocols.
    * **Session Management**: Provides protocols for applications to interact with session managers, allowing users to safely save their workspace state, restart programs, or shut down without losing functionality.
    * **Shared Resources**: Details the conventions for sharing input focus and color maps, ensuring that apps don't compete or lock each other out of basic system resources.
  - **EWMH (Extended window manager hints)**: EWMH stands for Extended Window Manager Hints. It is a standard for the X Window System that dictates how desktop environments, window managers, and applications communicate with one another. It builds upon older standards to let modern apps request specific behaviors (like going fullscreen or creating custom docks).

    The foundational standard for X11 communication (ICCCM) is notoriously complex and lacks the features we expect from modern operating systems, such as virtual desktops, docks, and window tabbing. EWMH bridges this gap by defining properties and messages (called "hints") that allow programs and the window manager to understand each other perfectly.

Under EWMH, the window manager exposes specific pieces of information on the root window, and applications use those to request states or react to desktop events. Key functions include:

* **Window Types & States**: Apps can declare what they are (e.g., a normal app, a dock, a splash screen, or a utility tool) or request states like _NET_WM_STATE_FULLSCREEN or _NET_WM_STATE_MODAL.
* **Virtual Desktops**: It defines protocols for managing workspaces, including how many virtual screens exist and which desktop a specific window lives on.
* **Window Tabbing & Grouping**: It specifies how windows can be logically grouped together (often seen in advanced tiling or tabbed window managers).
* **Desktop Properties**: It standardizes the retrieval of desktop dimensions, work areas (reserving space for taskbars), and active window tracking.

Developers working directly with window operations rely on specific EWMH messages (denoted by the **_NET** prefix):

* **_NET_ACTIVE_WINDOW**: Tells the window manager to give focus to a specific window.
* **_NET_CLOSE_WINDOW**: Requests a graceful closure of an application window.
* **_NET_WM_PING**: Allows the window manager to check if an application has stopped responding.

Note: EWMH is not standalone in the strict compliance sense. The EWMH spec explicitly says it builds on ICCCM, and that window managers and clients that aim to fulfill EWMH must also adhere to ICCCM, except where EWMH explicitly modifies ICCCM behavior.

- **Opaque structures**: An opaque struct is a design pattern in C where a structure is declared but its internal fields are hidden from the user. The user only interacts with the struct via pointers and a provided set of API functions, making it the primary way to achieve encapsulation and object-oriented ”data hiding” in C.

  A **transparent (exposed) struct** is the opposite. The fields are fully visible to the user.

- **Visual types**: On some display hardware, it may be possible to deal with color resources in more than one way. For example, you may be able to deal with a screen of either 12-bit depth with arbitrary mapping of pixel to color (pseudo-color) or 24-bit depth with 8 bits of the pixel dedicated to each of red, green, and blue. These different ways of dealing with the visual aspects of the screen are called visuals. For each screen of the display, there may be a list of valid visual types supported at different depths of the screen. Because default windows and visual types are defined for each screen, most simple applications need not deal with this complexity.

  Xlib uses an opaque Visual structure that contains information about the possible color mapping. The visual utility functions use an **XVisualInfo** structure to return this information to an application. The members of this structure pertinent to this discussion are class, red_mask, green_mask, blue_mask, bits_per_rgb, and colormap_size. The class member specified one of the possible visual classes of the screen and can be **StaticGray, StaticColor, TrueColor, GrayScale, PseudoColor,** or **DirectColor.**

A pixel value in X11 is usually just an integer. The Visual tells Xlib and the X server what that integer means. For example, does the integer directly encode red/green/blue components? Or is it an index into a colormap? Are the colors fixed, or can the client modify them?

The **Visual** describes the color interpretation model available for windows and pixmaps of a certain depth.

```
typedef struct {
    Visual *visual;
    VisualID visualid;
    int screen;
    int depth;
    int class;
    unsigned long red_mask;
    unsigned long green_mask;
    unsigned long blue_mask;
    int colormap_size;
    int bits_per_rgb;
} XVisualInfo;
```

There is no direct "convert Visual* to XVisualInfo" function, but you can get the VisualID from the Visual *, then ask Xlib for the matching XVisualInfo.

```
VisualID XVisualIDFromVisual(Visual* visual);
XVisualInfo* XGetVisualInfo(
    Display* display,
    long vinfo_mask,
    XVisualInfo* vinfo_template,
    int* nitems_return
);
```

- vinfo_mask Specifies the visual mask value.
- vinfo_template Specifies the visual attributes that are to be used in matching the visual structures.
- nitems_return Returns the number of matching visual structures.

Regarding the vinfo_mask: In Xlib, vinfo_mask is a bitwise mask value used as an argument in the XGetVisualInfo function to specify which fields of an XVisualInfo template structure the X server should evaluate when searching for available window visuals. When you use XGetVisualInfo, you pass a template structure containing the display criteria you want. The vinfo_mask tells the function exactly which of those template fields matter for your query

- **VisualNoMask**: No fields are matched (returns all available visuals)
- **VisualIDMask**: Matches the explicit visualid
- **IDVisualScreenMask**: Matches the specified screen
- **numberVisualDepthMask**: Matches the pixel bit depth (e.g., 24-bit, 32-bit color)

- **VisualClassMask**: Matches the color class (e.g., TrueColor, PseudoColor)
- **VisualRedMaskMask**: Matches the red_mask bit
- **positionsVisualGreenMaskMask**: Matches the green_mask bit
- **positionsVisualBlueMaskMask**: Matches the blue_mask bit
- **positionsVisualColormapSizeMask**: Matches the total entries in the colormap_size
- **VisualBitsPerRGBMask**: Matches the bits_per_rgb value
- **VisualAllMask**: Combines all masks to check every field in the structure

For example,

```
XVisualInfo template;
template.screen = screen;
template.depth = 32;
template.class = TrueColor;

int nitems;
XVisualInfo *matches = XGetVisualInfo(
    dpy,
    VisualScreenMask  VisualDepthMask  VisualClassMask,
    &template,
    &nitems
);
```

- **Class**: The class field tells you what kind of color model the visual uses. There are six visual classes
  1. **StaticGray**: A StaticGray visual represents fixed grayscale colors. The pixel value selects a gray value from a read-only colormap.

     You cannot change what gray value a pixel maps to. This is mostly relevant to old or limited displays.
  2. **GrayScale**: A GrayScale visual is like StaticGray, except the colormap entries can be changed. The colors are grayscale, but the client may allocate or change entries.
  3. **StaticColor**: A StaticColor visual uses a fixed color colormap. The pixel value indexes into a predefined table of colors. The client cannot redefine those colors.
  4. **PseudoColor**: A PseudoColor visual uses a modifiable colormap. For example, pixel value 42 does not inherently mean red or blue. It means "look at entry 42 in the colormap."

     The client may allocate/change colormap entries, depending on availability. This was common on older 8-bit indexed-color displays.
  5. **TrueColor**: A TrueColor visual means the pixel value directly contains RGB components. For example, with a common 24-bit / 32-bit visual.

     [pixel bits]: [[red bits] [green bits] [blue bits]]

     The masks say where the components are stored. For example,

     ```
     red_mask   = 0x00ff0000
     green_mask = 0x0000ff00
     blue_mask  = 0x000000ff
     ```

So a pixel value like

$$0x00ff0000.$$

means full red, zero green, zero blue.

In TrueColor, the mapping from component value to actual intensity is fixed. You cannot redefine what "red component 255" means. This is the usual modern visual class.

6. **DirectColor**: DirectColor is similar to TrueColor because the pixel value contains separate red, green, and blue fields.

   But instead of those fields directly giving final intensities, each component indexes into a separate modifiable lookup table.

The main distinction is

* **Static* classes**: colormap entries are fixed/read-only
* **Non-static classes**: colormap entries can be changed by the client
* **TrueColor / DirectColor**: pixel value contains separate RGB component fields
* **PseudoColor / StaticColor / GrayScale / StaticGray**: pixel value indexes into a colormap

– **red_mask, green_mask, blue_mask**: Described above in the *class* explanation.

– **bits_per_rgb**: Tells you the number of significant bits used for each RGB value in the colormap. For example, if

```
bits_per_rgb = 8;
```

then each color component has 8 meaningful bits, giving values from 0 to 255.

– **colormap_size**: The number of entries available in the colormap for that visual.

For example, an 8-bit PseudoColor visual might have:

```
colormap_size = 256;
```

because 8-bit pixels can index 256 possible entries.

For TrueColor, the meaning is slightly different. It usually refers to the number of entries per primary color map, not the total number of possible RGB colors.

So for a 24-bit TrueColor visual with 8 bits per channel, you might see:

```
colormap_size = 256;
```

Even though the visual can represent approximately

$$256^3 = 16777216$$

colors.

Note: An X11 client cannot define a custom `Visual`. A Visual is advertised by the X server as part of the screen's capabilities. When your program connects to the X server, the server tells the client which visuals exist for each screen. Your program can then choose from those visuals, but it cannot create a new one.

- **Top-level windows**: A top-level window is any window that is a direct subwindow of your screen's root window and is independently managed by your desktop's window manager.
- **Window attributes**: All **InputOutput** windows have a border width of zero or more pixels, an optional background, an event suppression mask (which suppresses propagation of events from children), and a property list. The window border and background can be a solid color or a pattern, called a tile. All windows except the root have a parent and are clipped by their parent. If a window is stacked on top of another window, it obscures that other window for the purpose of input.

  If a window has a background (almost all do), it obscures the other window for purposes of out- put. Attempts to output to the obscured area do nothing, and no input events (for example, pointer motion) are generated for the obscured area.

  Both **InputOutput** and **InputOnly** windows have the following common attributes, which are the only attributes of an **InputOnly** window:

  – win-gravity
  – event-mask
  – do-not-propagate-mask
  – override-redirect
  – cursor

  If you specify any other attributes for an **InputOnly** window, a **BadMatch** error results.

  InputOnly windows are used for controlling input events in situations where **InputOutput** windows are unnecessary. **InputOnly** windows are invisible; can only be used to control such things as cursors, input event generation, and grabbing; and cannot be used in any graphics requests. Note that **InputOnly** windows cannot have **InputOutput** windows as inferiors

  Windows have borders of a programmable width and pattern as well as a background pattern or tile. Pixel values can be used for solid colors. The background and border pixmaps can be destroyed immediately after creating the window if no further explicit references to them are to be made. The pattern can either be relative to the parent or absolute. If **ParentRelative,** the parent's background is used.

  When windows are first created, they are not visible (not mapped) on the screen. Any output to a window that is not visible on the screen and that does not have backing store will be discarded. An application may wish to create a window long before it is mapped to the screen. When a window is eventually mapped to the screen (using **XMapWindow),** the X server generates an Expose event for the window if backing store has not been maintained.

A window manager can override your choice of size, border width, and position for a top-level window. Your program must be prepared to use the actual size and position of the top window. It is not acceptable for a client application to resize itself unless in direct response to a human command to do so. Instead, either your program should use the space given to it, or if the space is too small for any useful work, your program might ask the user to resize the window. The border of your top-level window is considered fair game for window managers.

To set an attribute of a window, set the appropriate member of the **XSetWindowAttributes** structure and OR in the corresponding value bitmask in your subsequent calls to **XCreateWindow** and **XChangeWindowAttributes,** or use one of the other convenience functions that set the appropriate attribute. The symbols for the value mask bits and the **XSetWindowAttributes** structure are:

```
0  /* Window attribute value mask bits */
1  #define CWBackPixmap        (1L<<0)
2  #define CWBackPixel         (1L<<1)
3  #define CWBorderPixmap      (1L<<2)
4  #define CWBorderPixel       (1L<<3)
5  #define CWBitGravity        (1L<<4)
6  #define CWWinGravity        (1L<<5)
7  #define CWBackingStore      (1L<<6)
8  #define CWBackingPlanes     (1L<<7)
9  #define CWBackingPixel      (1L<<8)
10 #define CWOverrideRedirect  (1L<<9)
11 #define CWSaveUnder         (1L<<10)
12 #define CWEventMask         (1L<<11)
13 #define CWDontPropagate     (1L<<12)
14 #define CWColormap          (1L<<13)
15 #define CWCursor            (1L<<14)
```

```c
typedef struct {
    Pixmap background_pixmap;       /* background, None, or
    ↪  ParentRelative */
    unsigned long background_pixel; /* background pixel */
    Pixmap border_pixmap;           /* border of the window
    ↪  or CopyFromParent */
    unsigned long border_pixel;     /* border pixel value */
    int bit_gravity;                /* one of bit gravity
    ↪  values */
    int win_gravity;                /* one of the window
    ↪  gravity values */
    int backing_store;              /* NotUseful, WhenMapped,
    ↪  Always */
    unsigned long backing_planes;   /* planes to be preserved
    ↪  if possible */
    unsigned long backing_pixel;    /* value to use in
    ↪  restoring planes */
    Bool save_under;                /* should bits under be
    ↪  saved? (popups) */
    long event_mask;                /* set of events that
    ↪  should be saved */
    long do_not_propagate_mask;     /* set of events that
    ↪  should not propagate */
    Bool override_redirect;         /* boolean value for
    ↪  override_redirect */
    Colormap colormap;              /* color map to be
    ↪  associated with window */
    Cursor cursor;                  /* cursor to be displayed
    ↪  (or None) */
} XSetWindowAttributes;
```

- **background_pixmap**: Background_pixmap is a pixmap resource used as a tiled background. It can be:

```
None
ParentRelative
some_pixmap
```

  - **None:** None means the window has no automatic background. X will not repaint the background for you.
  - **ParentRelative**: ParentRelative means the window uses the parent window's background. This is only valid for windows with the same depth as the parent.
  - **real pixmap**: A real pixmap means the server tiles that pixmap across the window background.
- **long background_pixel**: background_pixel is simpler: it fills the background with one pixel value. The meaning of that pixel value depends on the window's colormap and visual. In a TrueColor visual, it usually encodes RGB bits. In PseudoColor, it is an index into a colormap.
- **border_pixmap**: border_pixmap uses a pixmap as the border pattern. It can also be CopyFromParent in some creation contexts.
- **long border_pixel**: border_pixel uses a single pixel value for the border color.

The border is the X window border specified by the $\text{border}_{w}idthargumenttoXCreateWindow.Manymoder$ $sideborders fortop - levelwindowsbecausetheydrawtheirowndecorations.$

– **bit_gravity**: Bit gravity controls what happens to the contents of the window when the window is resized.

For example, suppose your window gets larger. Should the old pixels stay in the upper-left corner? Center? Bottom-right? Or should X discard them?

Common values include

```
ForgetGravity
NorthWestGravity
NorthGravity
NorthEastGravity
WestGravity
CenterGravity
EastGravity
SouthWestGravity
SouthGravity
SouthEastGravity
StaticGravity
```

So,

```
attrs.bit_gravity = NorthWestGravity;
```

This means that when the window is resized, X tries to keep the old pixel contents attached to the upper-left corner.

ForgetGravity means X does not preserve the old bits. You should repaint when you receive an Expose event.

In modern applications, you usually repaint explicitly instead of relying heavily on bit gravity.

– **win_gravity**: Window gravity controls what happens to a child window's position when its parent is resized.

```
attrs.win_gravity = SouthEastGravity;
```

means that when the parent resizes, X tries to keep the child attached to the parent's bottom-right region.

This matters mostly for child-window layout. It is not usually used as the primary layout mechanism in modern GUI toolkits.

A win-gravity of **UnmapGravity** is like **NorthWestGravity** (the window is not moved), except the child is also unmapped when the parent is resized, and an **UnmapNotify** event is generated.

– **backing_store**: Backing store asks the *X* server to preserve obscured or unmapped window contents, so that when the window becomes visible again, the server may restore pixels without requiring the client to repaint.

Possible values are

```
0 NotUseful
1 WhenMapped
2 Always
```

* NotUseful means you do not request backing store.
* WhenMapped means the server may preserve contents while the window is mapped.
* Always means the server should try to preserve contents more aggressively.

However, this is only a hint. The server is not required to honor it. Modern systems often ignore backing store or implement it differently because compositors already keep window buffers.

– **backing_planes**: backing_planes specifies which bit planes should be preserved.

– **backing_pixel**: backing_pixel gives a fallback pixel value for restoring planes that were not preserved.

Most modern Xlib programs simply handle Expose events and repaint instead of relying on backing store.

– **save_under**: save_under is a hint to the server that it should save the pixels underneath this window while it is visible.

This was intended for temporary popup windows, menus, tooltips, and similar small transient windows.

Like backing store, this is only a hint. The server may ignore it.

– **event_mask**: This selects which events your client wants to receive for this window.

Some common masks are

```
0  ExposureMask
1  KeyPressMask
2  KeyReleaseMask
3  ButtonPressMask
4  ButtonReleaseMask
5  PointerMotionMask
6  EnterWindowMask
7  LeaveWindowMask
8  StructureNotifyMask
9  SubstructureNotifyMask
10 SubstructureRedirectMask
11 PropertyChangeMask
12 FocusChangeMask
```

For example, if you do not select **KeyPressMask,** then your client will not receive **KeyPress** events for that window.

Some event masks are exclusive. For example, only one client can select **SubstructureRedirectMask** on a given window. That is how a window manager claims control over the root window's child-window management.

- **do_not_propagate_mask**: This controls which input events should not propagate upward to ancestor windows.

  In X, some events can propagate from a child window to its parent if no client selected them on the child.

  For example, suppose a button press occurs in a child window. If nobody selected **ButtonPressMask** on that child, the event may propagate to the parent.

  The **do_not_propagate_mask** can stop that propagation.
- **override_redirect**: This tells the X server whether the window manager should be bypassed for this window. If true, then the window manager normally does not manage the window. That means no decorations, no placement policy, no focus policy, no normal reparenting, and no window-manager intervention.
- **colormap**: This is the colormap associated with the window.

  A colormap maps pixel values to actual colors, depending on the visual class.

  For common TrueColor visuals, you usually use the default colormap:

  But if you create a window using a non-default visual, you often need to create and assign a matching colormap:

```
0 Colormap cmap = XCreateColormap(display, root, visual,
  ↪  AllocNone);
1 attrs.colormap = cmap;
```

  This is especially important when creating windows with 32-bit ARGB visuals for transparency. The window's visual and colormap must be compatible.
- **cursor**: This specifies the cursor shown when the pointer is inside the window.

  You can use a cursor created with functions like:

```
0 XCreateFontCursor
1 XCreatePixmapCursor
2 XcursorLibraryLoadCursor
```

  If the cursor is None, the cursor is inherited from the parent window.

The following lists the defaults for each window attribute and indicates whether the attribute is applicable to **InputOutput** and **InputOnly** windows:

Attribute	Default	InputOutput	InputOnly
background-pixmap	None	Yes	No
background-pixel	Undefined	Yes	No
border-pixmap	CopyFromParent	Yes	No
border-pixel	Undefined	Yes	No
bit-gravity	ForgetGravity	Yes	No
win-gravity	NorthWestGravity	Yes	Yes
backing-store	NotUseful	Yes	No
backing-planes	All ones	Yes	No
backing-pixel	zero	Yes	No
save-under	False	Yes	No
event-mask	empty set	Yes	Yes
do-not-propagate-mask	empty set	Yes	Yes
override-redirect	False	Yes	Yes
colormap	CopyFromParent	Yes	No
cursor	None	Yes	Yes

- **Background Attribute**: Only **InputOutput** windows can have a background. You can set the background of an **InputOutput** window by using a pixel or a pixmap.
- **Pixel centers**: In graphics, to **coincide with pixel centers** means that some geometric coordinate, such as a line, point, or sample location, falls exactly at the center of a pixel, not on the boundary between pixels.
- **Creating Windows**: Xlib provides basic ways for creating windows, and toolkits often supply higher-level functions specifically for creating and placing top-level windows, which are discussed in the appropriate toolkit documentation. If you do not use a toolkit, however, you must provide some standard information or hints for the window manager by using the Xlib inter-client communication functions

  If you use Xlib to create your own top-level windows (direct children of the root window), you must observe the following rules so that all applications interact reasonably across the different styles of window management:

  – You must never fight with the window manager for the size or placement of your top-level window.

  – You must be able to deal with whatever size window you get, even if this means that your application just prints a message like ‘‘Please make me bigger’’ in its window.

  – You should only attempt to resize or move top-level windows in direct response to a user request. If a request to change the size of a top-level window fails, you must be prepared to live with what you get. You are free to resize or move

the children of top-level windows as necessary. (Toolkits often have facilities for automatic relayout.)

– If you do not use a toolkit that automatically sets standard window properties, you should set these properties for top-level windows before mapping them.

XCreateWindow is the more general function that allows you to set specific window attributes when you create a window. **XCreateSimpleWindow** creates a window that inherits its attributes from its parent window.

The X server acts as if **InputOnly** windows do not exist for the purposes of graphics requests, exposure processing, and **VisibilityNotify** events. An **InputOnly** window cannot be used as a drawable (that is, as a source or destination for graphics requests). **InputOnly** and **InputOutput** windows act identically in other respects (properties, grabs, input control, and so on). Extension packages can define other classes of windows.

To create an unmapped window and set its window attributes, use **XCreateWindow**

```
Window XCreateWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    int depth,
    unsigned int class,
    Visual* visual,
    unsigned long valuemask,
    XSetWindowAttributes* attributes
)
```

– **display**: Specifies the connection to the X server.

– **parent**: Specifies the parent window.

– **x, y**: Specify the x and y coordinates, which are the top-left outside corner of the created window's borders and are relative to the inside of the parent window's borders.

– **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.

– **border_width**: Specifies the width of the created window's border in pixels.

– **depth**: Specifies the window's depth. A depth of **CopyFromParent** means the depth is taken from the parent.

– **class**: Specifies the created window's class. You can pass **InputOutput, InputOnly,** or **CopyFromParent.** A class of **CopyFromParent** means the class is taken from the parent.

– **visual**: Specifies the visual type. A visual of **CopyFromParent** means the visual type is taken from the parent.

– **valuemask**: Specifies which window attributes are defined in the attributes argument. This mask is the bitwise inclusive OR of the valid attribute mask bits. If valuemask is zero, the attributes are ignored and are not referenced.

– **attributes**: Specifies the structure from which the values (as specified by the value mask) are to be taken. The value mask should have the appropriate bits set to indicate which attributes have been set in the structure.

The **XCreateWindow** function creates an unmapped subwindow for a specified parent window, returns the window ID of the created window, and causes the X server to generate a **CreateNotify** event. The created window is placed on top in the stacking order with respect to siblings.

The coordinate system has the X axis horizontal and the Y axis vertical with the origin [0, 0] at the upper-left corner. Coordinates are integral, in terms of pixels, and coincide with pixel centers. Each window and pixmap has its own coordinate system. For a window, the origin is inside the border at the inside, upper-left corner.

The border_width for an **InputOnly** window must be zero, or a **BadMatch** error results. For class **InputOutput,** the visual type and depth must be a combination supported for the screen, or a **BadMatch** error results. The depth need not be the same as the parent, but the parent must not be a window of class **InputOnly,** or a **BadMatch** error results. For an **InputOnly** window, the depth must be zero, and the visual must be one supported by the screen. If either condition is not met, a **BadMatch** error results. The parent window, however, may have any depth and class. If you specify any invalid window attribute for a window, a **BadMatch** error results.

- **Creating simple windows**: The created window is not yet displayed (mapped) on the user's display. To display the window, call **XMapWindow.** The new window initially uses the same cursor as its parent. A new cursor can be defined for the new window by calling **XDefineCursor.** The window will not be visible on the screen unless it and all of its ancestors are mapped and it is not obscured by any of its ancestors.

XCreateWindow can generate **BadAlloc, BadColor, BadCursor, BadMatch, BadPixmap, BadValue,** and **BadWindow** errors.

To create an unmapped **InputOutput** subwindow of a given parent window, use **XCreateSimpleWindow**

```
Window XCreateSimpleWindow(
    Display* display,
    Window parent,
    int x, y,
    unsigned int width, height,
    unsigned int border_width,
    unsigned long border,
    unsigned long background
)
```

– **display**: Specifies the connection to the X server.

– **parent**: Specifies the parent window.

– **x, y**: Specify the x and y coordinates, which are the top-left outside corner of the new window's borders and are relative to the inside of the parent window's borders.

- **width, height**: Specify the width and height, which are the created window's inside dimensions and do not include the created window's borders. The dimensions must be nonzero, or a **BadValue** error results.
- **border_width**: Specifies the width of the created window's border in pixels.
- **border**: Specifies the border pixel value of the window.
- **background**: Specifies the background pixel value of the window.

The **XCreateSimpleWindow** function creates an unmapped **InputOutput** subwindow for a specified parent window, returns the window ID of the created window, and causes the X server to generate a **CreateNotify** event. The created window is placed on top in the stacking order with respect to siblings. Any part of the window that extends outside its parent window is clipped. The border_width for an **InputOnly** window must be zero, or a **BadMatch** error results. **XCreateSimpleWindow** inherits its depth, class, and visual from its parent. All other window attributes, except background and border, have their default values.

XCreateSimpleWindow can generate **BadAlloc, BadMatch, BadValue,** and **BadWindow** errors.

- **Destroying Windows**: Xlib provides functions that you can use to destroy a window or destroy all subwindows of a window.

  To destroy a window and all of its subwindows, use **XDestroyWindow.**

```
XDestroyWindow(Display* display, Window w)
```

The **XDestroyWindow** function destroys the specified window as well as all of its subwindows and causes the X server to generate a **DestroyNotify** event for each window. The window should never be referenced again. If the window specified by the w argument is mapped, it is unmapped automatically. The ordering of the **DestroyNotify** events is such that for any given window being destroyed, **DestroyNotify** is generated on any inferiors of the window before being generated on the window itself. The ordering among siblings and across **subhierarchies** is not otherwise constrained. If the window you specified is a root window, no windows are destroyed. Destroying a mapped window will generate Expose events on other windows that were obscured by the window being destroyed.

XDestroyWindow can generate a **BadWindow** error

To destroy all subwindows of a specified window, use **XDestroySubwindows**

```
XDestroySubwindows(Display* display, Window w)
```

The **XDestroySubwindows** function destroys all inferior windows of the **specified** window, in bottom-to-top stacking order. It causes the X server to generate a **DestroyNotify** event for each window. If any mapped subwindows were actually destroyed, **XDestroySubwindows** causes the X server to generate **Expose** events on the specified window. This is much more efficient than deleting many windows one at a time because much of the work need be performed only once for all of the windows, rather than for each window. The subwindows should never be referenced again.

XDestroySubwindows can generate a **BadWindow** error.

- **Mapping windows**:

4. Window information functions

5. Pixmap and cursor functions

6. Color management functions

7. Graphics context functions

8. Graphics functions

9. Window and session manager functions

10. Events

11. Event handling functions

12. Input device functions